Hardware/Software Co-Design for Image Processing Based on RISC-V Architecture

Embedded Vision Deployment and Board Bring-Up

Report Type: Technical Briefing

Scope: Board Bring-Up, Networking, VM Environment, and License Activation

Date: March 7, 2026

Context: This report summarizes the accomplishments and engineering workflows developed during the Spring 2026 Independent Study. It emphasizes board acceptance, reproducible flashing, UART-based diagnosis, network authentication adaptation, and a virtual-machine-based vendor toolchain workflow.

Executive Summary

This report presents an engineering-oriented review of a development board bring-up and embedded vision deployment workflow. It covers hardware acceptance, system re-flashing, UART debugging, U-Boot recovery, SSH and file transfer setup, wired-network authentication adaptation, VM-based Libero deployment, and license activation. The core principle is to turn a fragmented setup process into a repeatable, transferable, and scalable engineering workflow.

Contents

1 Board Reception and Basic Hardware Verification

After receiving the board, the first step is to confirm its basic hardware characteristics. The current platform provides a 1 GHz ARM Cortex-A8 processor, an SGX 3D Graphics Engine, a NEON floating-point accelerator, and two 32-bit 200 MHz PRUs. This means the board is capable of handling not only basic embedded control tasks, but also lightweight graphics processing, floating-point computation, and real-time co-processing. For that reason, it is well suited for peripheral control as well as lightweight vision algorithms and edge-side application validation.

Before power-up and image burning, a standardized acceptance check should be completed. This includes inspecting the board for physical damage such as dents or solder defects, verifying that the power adapter, UART cable, USB Type-C cable, network cable, and storage card are all present, and powering on the board with only the necessary peripherals connected so that the power indicator can be observed for stability. It is also recommended to record the board serial number, batch number, and firmware version for future reference. If possible, photographs of the front and back of the board should be archived as baseline evidence so that physical damage is not mistaken for a software problem later.

2 System Re-Flashing

Once the hardware has been confirmed, the next task is to re-flash the system image. To ensure consistency across future environments, the official image is usually downloaded from the vendor site and written using one or more tools. In this practice, the official flashing utility was not stable enough in terms of compatibility, and balenaEtcher-2.1.4.Setup also produced unsatisfactory results. In the end, `rufus-4.11_x86` successfully completed the image write.

This shows that the best flashing tool in an actual engineering environment is not always the vendor-provided one; what matters more is choosing the tool with the highest success rate and the most predictable behavior in the current setup. To reduce the risk of difficult-to-trace issues such as "flashing succeeded but the system behaves abnormally," the image should be verified before burning. The recommended process is to record the version number, release time, and hash value from the official page, such as SHA256, then compute the local hash of the downloaded image and compare it byte-for-byte with the official value. Only after the values match should the write process begin.

Rufus parameters should also be kept as stable as possible: make sure the target device is selected correctly and not mistaken for a host USB drive, choose the partition scheme according to the official instructions or keep the image default if no explicit instruction exists, use the recommended write mode when compatibility warnings appear, and enable

post-write verification so that storage-card defects or write corruption can be detected early. Capturing the final working configuration in the documentation can significantly reduce repeated trial and error later.

3 Initial Connection and UART Debugging

After the image is burned, the first common issue is not usually a boot failure, but an inability to log in via SSH. At this point, the most effective debugging method is to switch to the UART debug port and use the serial log to determine where the boot sequence is failing. Once the board is connected to the host through USB, the corresponding COM port can be confirmed in Windows Device Manager and then opened with PuTTY.

The serial session should be configured as Serial mode, the COM number should be set to the actual port, for example COM3, the baud rate should be fixed at 115200, and flow control should be set to None to avoid garbled output and input problems. If a four-wire serial cable is used, the cross-connection rule must be followed carefully: black to GND, green RXD to board TX, white TXD to board RX, and red VCC should be left unconnected to avoid accidental power injection that could damage the board. If no output appears after connection, the first thing to try is swapping the green and white wires or rechecking the COM port selection.

3.1 Beginner UART Checklist (Step-by-Step)

To ensure a reliable connection and avoid common setup mistakes during the project, I established the following sequence:

1. Connect the board to the PC with the USB-to-UART cable according to the pinout definitions in Table 1.

Wire Color	Signal	Label	Description & Connection
Black ●	GND	Ground	Connect to board **GND**
Green ●	RXD	Receive Data	Connect to board **TXD**
White ○	TXD	Transmit Data	Connect to board **RXD**
Red ●	VCC	Power	**Leave Disconnected** (Avoid burning the board)

Table 1: USB-to-UART Cable Wiring Specifications

2. Open `Device Manager` on Windows and Expand `Ports (COM & LPT)` and note the detected port number, such as `COM3`.

3. Open PuTTY and set `Connection type = Serial`, `Serial line = COMx`, and `Speed = 115200`., and Navigate to `Connection -> Serial` and set `Flow control = None`.

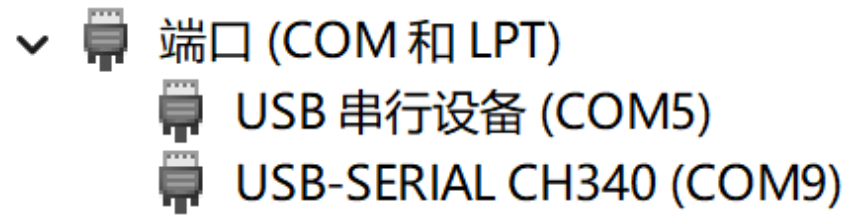

Figure 1: Device Manager

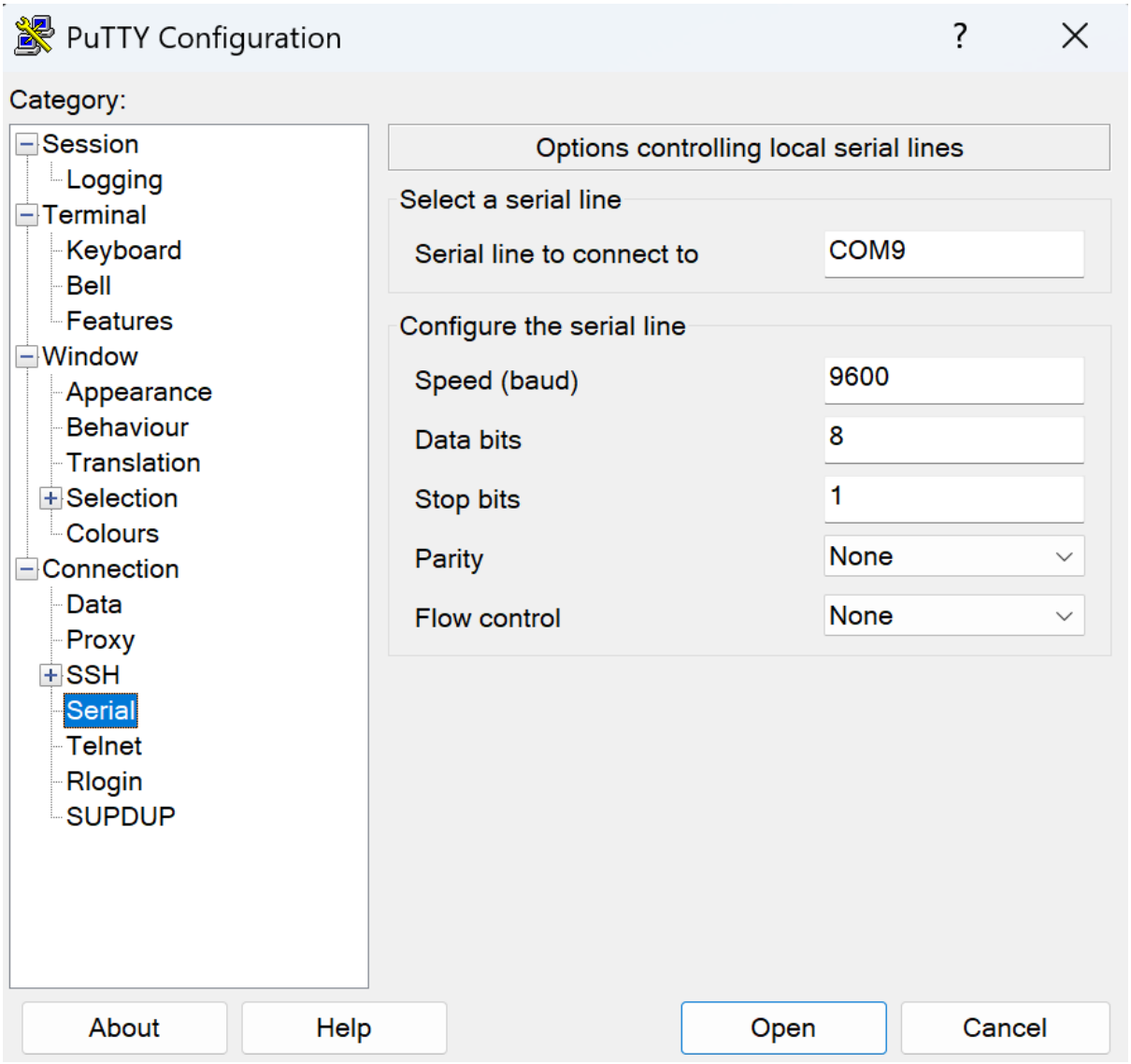

Figure 2: PuTTY

4. Click `Open`; then reset the board and confirm that boot logs appear immediately.

If no output appears, first swap RX/TX wiring (green and white wires), then re-check the COM port. If output is garbled, verify that baud rate is still `115200` and flow control remains `None`.

For typical issues such as no terminal output, garbled output, logs appearing but no

command input, or intermittent disconnection, the troubleshooting process can also be organized by symptom, cause, and action. Typical checks include baud-rate mismatch, flow control being enabled, serial-tool conflicts, and unstable USB power delivery. Serial logs should ideally be saved as text with timestamps so that boot-stage differences before and after an error can be compared later.

Serial logs further show that the system can boot successfully, but on the first boot it requires the user password to be reset. At the same time, the system state does not allow the stored authentication data to be modified directly, which traps the board in a loop of initialization and permission restriction. The system therefore cannot complete a normal login. To solve this, the automatic boot process must be interrupted and the U-Boot command line opened so that the board storage can be accessed and the configuration file edited.

In practice, the Reset button is pressed during boot, and when the serial terminal begins printing boot messages, any key such as Space or Enter is pressed quickly and repeatedly to break into the U-Boot prompt, which appears in a form similar to $BeagleV-Fire$. In U-Boot, *mmclist* can be used to inspect the storage devices. Usually $mmc0$ corresponds to the on-board eMMC and $mmc1$ corresponds to the external SD card. Next, USB mass storage mode must be enabled: select the MMC device and run *usbdmsc* so that the host sees the board storage as a removable disk.

Once mounted, the $sysconf.txt$ file in the root directory should be opened in a text editor. In the user-configuration section, remove the leading comment symbols from the relevant entries and change the preset *username* and *password* to the target account and password. After saving the file and safely ejecting the disk, reboot the board and log in using the updated credentials. Conceptually, this is a rescue procedure that uses the low-level bootloader environment to correct system configuration when the normal boot chain is constrained.

4 SSH Login Configuration

After account recovery, the board must be connected to the host network so that SSH, *scp*, $rsync$, and other remote tools can be used in daily work. This workflow uses a USB Type-C virtual network adapter: when the board is connected to the Windows host, a new virtual interface appears. The network is usually configured manually, with the gateway set to 192.168.7.1, the Windows virtual adapter set to 192.168.7.3, and the board address set to 192.168.7.2.

Once these values are configured, the board can be accessed with:

```
ssh beagle@192.168.7.2
```

To improve both stability and security, the default password should be changed immediately after the first login, $ssh - keygen$ should be used to generate a key pair, and key-based login should be preferred whenever possible. The board-side *sshd* service should also be set to start on boot so that the remote channel does not disappear after reboot. In a lab where multiple boards are used in parallel, assigning fixed hostnames and keeping an IP mapping table can help avoid misconnection and cross-wiring issues.

4.1 Terminal Connection and Management using MobaXterm

For serial communication and remote management of the board, **MobaXterm** was selected as the primary terminal emulator. Rather than relying on basic tools like PuTTY or Windows Terminal, MobaXterm provides an all-in-one solution that significantly streamlines the board bring-up and debugging workflow.

Its primary advantages in embedded development include:

- **All-in-One Protocol Support:** It seamlessly integrates Serial (UART), SSH, Telnet, and VNC within a single tabbed interface. This allows developers to monitor the local serial boot log in one tab while executing SSH commands over the network in another, without switching between different applications.
- **Integrated SFTP File Browser:** When an SSH session is established, MobaXterm automatically opens a graphical SFTP browser in the left sidebar that tracks the terminal's current directory. This enables intuitive drag-and-drop file transfers between the host PC and the board, eliminating the need for separate software like WinSCP.
- **Robust Session Management:** Debugging often requires repeatedly connecting to specific COM ports or IP addresses. MobaXterm allows users to save session parameters (such as the exact COM port, 115200 baud rate, usernames, and SSH keys) into a localized profile, allowing one-click reconnections.
- **Enhanced Readability and X11 Forwarding:** It features built-in syntax highlighting, making it much easier to spot errors, warnings, or specific keywords during dense U-Boot or Linux kernel boot logs. Additionally, its embedded X server allows for seamless forwarding of graphical applications if needed in later development stages.

By centralizing communication, file transfer, and log monitoring, MobaXterm reduces context switching and ensures a smoother interaction with the hardware during the bring-up phase.

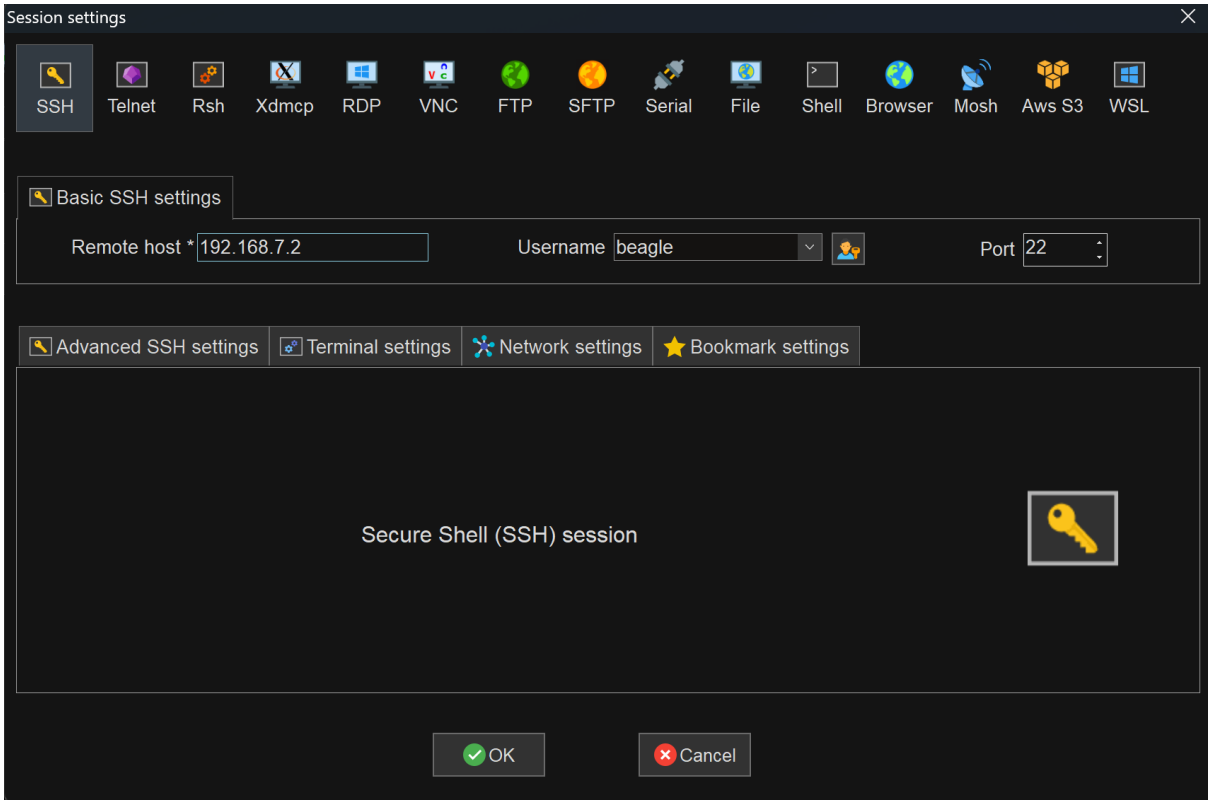

Figure 3: mobaXterm session creation

5 Data Transfer

Once remote login is available, file transfer becomes one of the core daily operations. For quick and general-purpose transfers of small files, *scp* is usually sufficient, for example when uploading a local file or recursively copying an entire directory. For larger transfers, repeated synchronization, or cases where resume support is required, *rsync* is a better choice because it supports incremental transfer, resumption, and higher synchronization efficiency.

For temporary text streams or data that should not be written to intermediate files, *ssh* combined with pipes is a lightweight solution. For example, a local file can be piped through *cat* and written directly to a remote path, or a directory can be compressed with *tar* and decompressed on the target side through *ssh*. If an FTP-like interactive workflow is preferred, $sftp$ can be used.

The practical selection rule is simple: use *scp* for one-off small file transfers, *rsync* for repeated directory synchronization, $ssh + tar$ for pipeline-based builds that should avoid intermediate files, and $sftp$ for interactive remote browsing and manual management. If the link is unstable, *rsync* can be extended with retries and log output, and the synchronization result can be recorded in the build log for later traceability.

5.1 Transfer Method Selection Guidance

For fast decision-making across different tasks, the following rule of thumb can be used:

- For temporary single-file transfer, prefer *scp* because it is concise and easy to use.
- For repeated directory synchronization, prefer *rsync* because it saves bandwidth and time.
- For build pipelines that should avoid intermediate files, prefer $ssh + tar$.
- For interactive browsing of remote directories, use $sftp$.

If the link quality is poor, *rsync* can be given retry logic and log output, and the synchronization result can be written to a build record for later tracing.

6 Wired Network Configuration and Authentication Adaptation

In wired-network bring-up, the problem is not just connectivity but authentication. In campus or enterprise networks, Ethernet access is often protected by portal authentication or MAC address binding. Because the development board cannot complete browser-based authentication like a PC, the common solution is to clone the MAC address of an authenticated Windows machine so that the board is recognized by the network as an already authenticated endpoint.

6.1 Step-by-Step: Get and Clone the Physical Address (MAC)

To make the process directly reproducible for beginners, follow the sequence below without skipping steps:

1. Plug the Ethernet cable into a Windows PC and complete the normal web portal login.
2. Open `cmd` via `Win + R`, then run:

```
ipconfig /all
```

3. Locate the active wired adapter and copy its `Physical Address`. The one represents the used WiFi.

```
Wireless LAN adapter Wi-Fi:
   Connection-specific DNS Suffix  . :
   Description . . . . . . . . . . . : Intel(R) Wi-Fi 6 AX201 160MHz
   Physical Address. . . . . . . . . : 9C-B1-50-9B-26-8E
   DHCP Enabled. . . . . . . . . . . : Yes
```

```
    Autoconfiguration Enabled . . . . : Yes
    Link-local IPv6 Address . . . . . : fe80::da23:1f71:99d6:7a55%9(
     Preferred)
    IPv4 Address. . . . . . . . . . . : 10.32.16.169(Preferred)
```

4. Convert the format if needed: Windows may show `C8-17-F5-2D-D1-3C`; on Linux commands use colons, i.e., `C8:17:F5:2D:D1:3C`.

5. Move the Ethernet cable from the PC to the board.

6. On the board, identify the wired interface:

```
ip link show
```

7. Replace `eth0` with your actual interface name and run:

```
sudo ip link set dev eth0 down
sudo ip link set dev eth0 address C8:17:F5:2D:D1:3C
sudo ip link set dev eth0 up
ip link show eth0
sudo dhclient eth0
ping -c 4 www.baidu.com
```

The workflow still follows modern Linux network management, using `ip` instead of the gradually deprecated `ifconfig`. To bypass authentication restrictions, the usual procedure is to first identify the physical address of the active wired adapter on the authenticated Windows host, and then clone it to the development board.

```
# Step 1: On the authenticated Windows host, retrieve the MAC address
ipconfig /all

# Step 2: Disconnect the cable from PC, connect to the board, and check
    interface
ip link show      # Usually eth0 or end0

# Step 3: Execute MAC spoofing sequence on the board
ip link set eth0 down
ip link set eth0 address 9C:B1:50:9B:26:8E  # Replace with the PC's MAC
    address
ip link set eth0 up

# Step 4: Renew IP and verify reachability
dhclient eth0
```

```
ping www.baidu.com
```

A successful setup should show two signals: first, `ip link show eth0` reports the new `link/ether` value you configured; second, `ping` returns normal replies such as `64 bytes from ... time=...` . If `dhclient` does not return an address, re-check whether the MAC came from an already authenticated PC interface and whether the interface name on the board is correct.

It should be noted that this change only applies during the current boot session; after reboot, the board will revert to its factory MAC address. If manual repetition is undesirable, a `systemd` service can be used to automate the process at boot.

```
# 1. Place your automation script and make it executable
chmod +x /usr/local/bin/mac-spoof.sh

# (The script should wait a few seconds after startup, change the
# interface address, and restore networking through dhclient/NetworkManager)

# 2. Create the systemd unit file
# Path: /etc/systemd/system/mac-spoof.service

# 3. Reload systemd manager configuration and enable the service
systemctl daemon-reload
systemctl enable mac-spoof.service
```

With this arrangement, the board automatically completes MAC cloning and network initialization on every power-up.

6.2 Automatic MAC Spoofing at Boot

To avoid repeated manual setup after every reboot, a *systemd* service can be used to trigger automatic MAC cloning and network initialization.

A helper script named $mac - spoof.sh$ can be created under $/usr/local/bin/$ with a copy-and-run block:

```
sudo bash -c 'cat > /usr/local/bin/mac-spoof.sh << "EOF"
#!/bin/bash
export PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

INTERFACE="eth0"
MAC="C8:17:F5:2D:D1:3C"

sleep 3
```

```
ip link set dev $INTERFACE down
ip link set dev $INTERFACE address $MAC
ip link set dev $INTERFACE up

if systemctl is-active --quiet systemd-networkd; then
  systemctl restart systemd-networkd
elif systemctl is-active --quiet NetworkManager; then
  systemctl restart NetworkManager
else
  dhclient $INTERFACE 2>/dev/null
fi
EOF'
```

Then grant execute permission:

```
sudo chmod +x /usr/local/bin/mac-spoof.sh
```

Create the service unit:

```
sudo bash -c 'cat > /etc/systemd/system/mac-spoof.service << "EOF"
[Unit]
Description=Auto Clone MAC Address and Get IP on Boot
After=network.target

[Service]
Type=oneshot
ExecStart=/usr/local/bin/mac-spoof.sh
RemainAfterExit=yes

[Install]
WantedBy=multi-user.target
EOF'
```

Enable and verify:

```
sudo systemctl daemon-reload
sudo systemctl enable mac-spoof.service
sudo systemctl start mac-spoof.service
systemctl status mac-spoof.service --no-pager
```

With this setup, the board automatically performs MAC cloning and network initialization on every boot.

7 Environment Configuration After Installation

Once the basic network and login environment is stable, the development environment itself can be prepared. If the system time is significantly off, package repositories, certificate verification, and software downloads may be rejected, so time correction should be performed first.

```
# Check the current system time
date

# Manually adjust the time if necessary (Format: YYYY-MM-DD HH:MM:SS)
sudo date -s "2026-04-25 14:30:00"
```

In addition to time correction, a minimal software initialization is recommended after installation so that later setup failures are less likely. Align the time zone and localization settings to avoid timestamp confusion in logs, update the package index, and create a clean project directory structure that separates source code, build artifacts, logs, and backups.

```
# Update package index and install common build & diagnostic tools
sudo apt update && sudo apt upgrade -y
sudo apt install -y git curl vim build-essential

# Create a standardized project directory structure
mkdir -p ~/projects/{src,build,logs,backups}
```

It is also a good idea to save these initialization steps as a script so that the environment can be reconstructed quickly after a reinstall.

After basic initialization, the larger deployment question must be addressed: how to organize older versions of Libero, MSS Configurator, and SoftConsole into an environment that is runnable, rollbackable, and reproducible. Because Libero is highly sensitive to the host operating system version and dependency stack, installing it directly on the host often leads to compatibility problems. A safer approach is to build a separate virtual machine environment with VMware so that development is isolated from the host system.

Before setting up the VM, it is recommended to create a Windows restore point on the host, for example named `Before_VMware_Install`, so that the system can be rolled back quickly if blue screens, driver conflicts, or bridge failures occur. VMware Workstation Pro or Player is recommended because it provides both environment isolation and snapshots, which are useful when patch installation, driver repair, or license configuration fails.

For optimal performance and compatibility, the VM should be configured with the following

baseline specifications:

- **OS**: Windows 10 Professional
- **CPU**: At least 4 cores
- **RAM**: 16 GB
- **Storage**: 150 GB of disk space
- **Network**: Bridged networking (to share the same LAN as the host and the board)

8 Virtual Machine Environment Setup

Bridged networking is often the most error-prone part of the VM stage. If bridged mode cannot be enabled or `vmnetbridge.dll` is reported missing, the first check should be whether Hyper-V, Virtual Machine Platform, or Windows Hypervisor Platform is enabled on the host. Core isolation and memory integrity should also be disabled in Windows Security because these features can occupy the low-level physical network driver and interfere with VMware bridging.

VMware can then be rerun as Administrator and repaired using the **Repair** option. If DLLs are still missing, the VMware installation directory must be specified manually, for example, `C:\Program Files (x86)\VMware\VMware Workstation`. It is also recommended not to use automatic bridging. Instead, bind `VMnet0` to a specific physical NIC (e.g., `Realtek USB GbE`), so that changes on the host network do not cause the VM interface to drift and affect license stability.

In the advanced network settings, bandwidth limits can also be configured as needed, for example by setting **Incoming transfer rate** to `20480 Kbps` and **Outgoing transfer rate** to `2048 Kbps`. All other parameters should remain at their defaults, and the **MAC Address** should not be reset casually because a license that is bound to the NIC will fail if the address changes.

9 Libero SoC Installation and License Activation

Before applying for the license, the Libero SoC software itself must be downloaded and installed. When obtaining the software from the Microchip website, it is highly recommended to select the **Web Installer** (online installer) rather than the standalone Full Installer. Because the toolchain is exceptionally large, downloading the entire standalone archive in one go often leads to corrupted files or Cyclic Redundancy Check (CRC) errors due to minor network fluctuations. The Web Installer is much more resilient as it downloads, verifies, and extracts components sequentially.

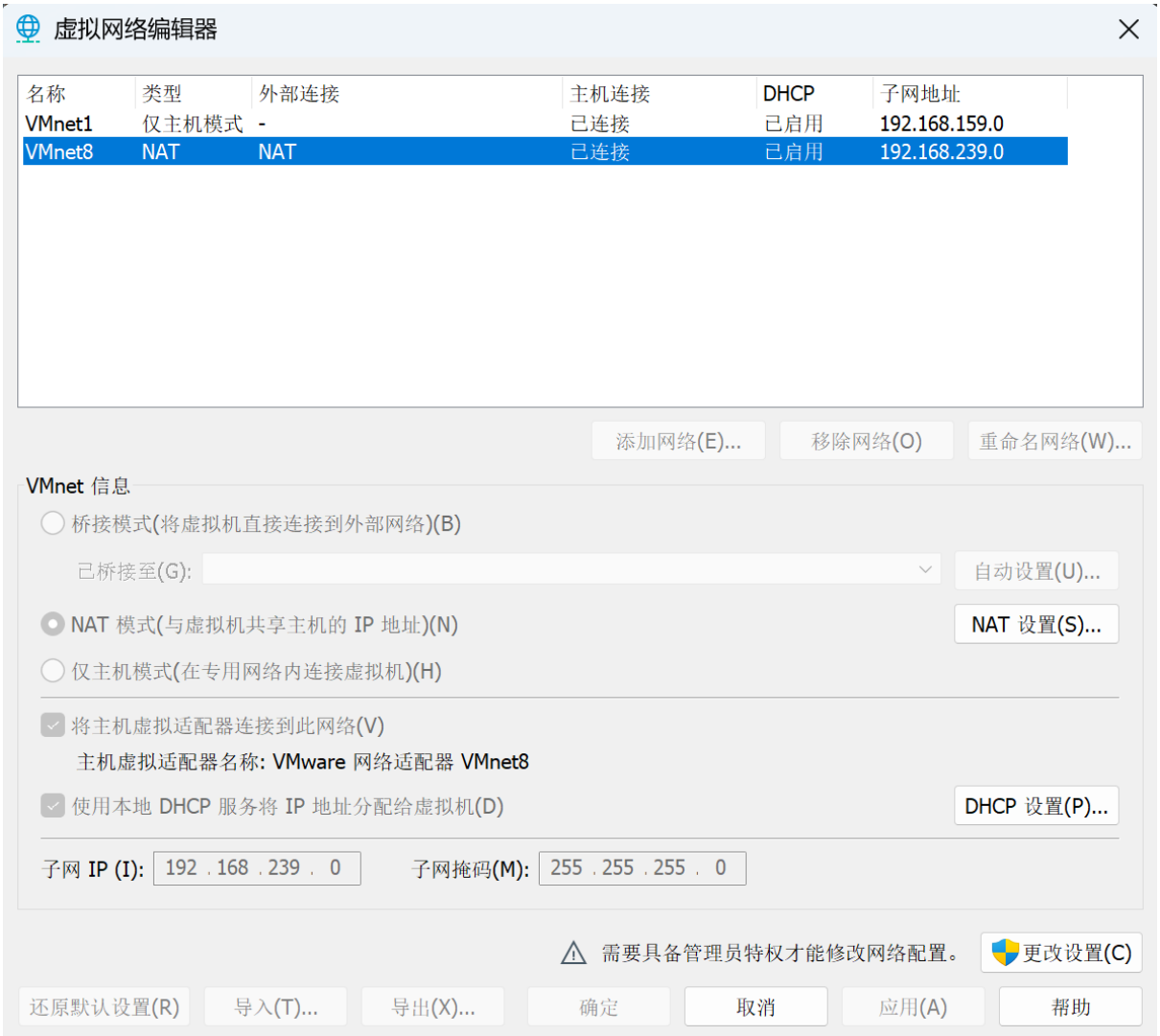

Figure 4: NAT Config

After the software installation is complete, the next step is to obtain and activate a Libero license. Following the usual procedure, open a command prompt in the VM to obtain the volume serial number:

```
# Check the volume serial number for the C: drive
vol c:
```

This DiskID usually appears in the form `XXXX-XXXX` and is the key identifier for the Silver License application.

Then log in to the official Microchip website, open the Libero licensing page, navigate to **Request Free License and Manage Licenses**, choose **Libero Silver 1 Yr DiskID NL License**, enter the recorded DiskID, and submit the request. After roughly ten minutes, the resulting `License.dat` file can usually be retrieved by email or directly from the website.

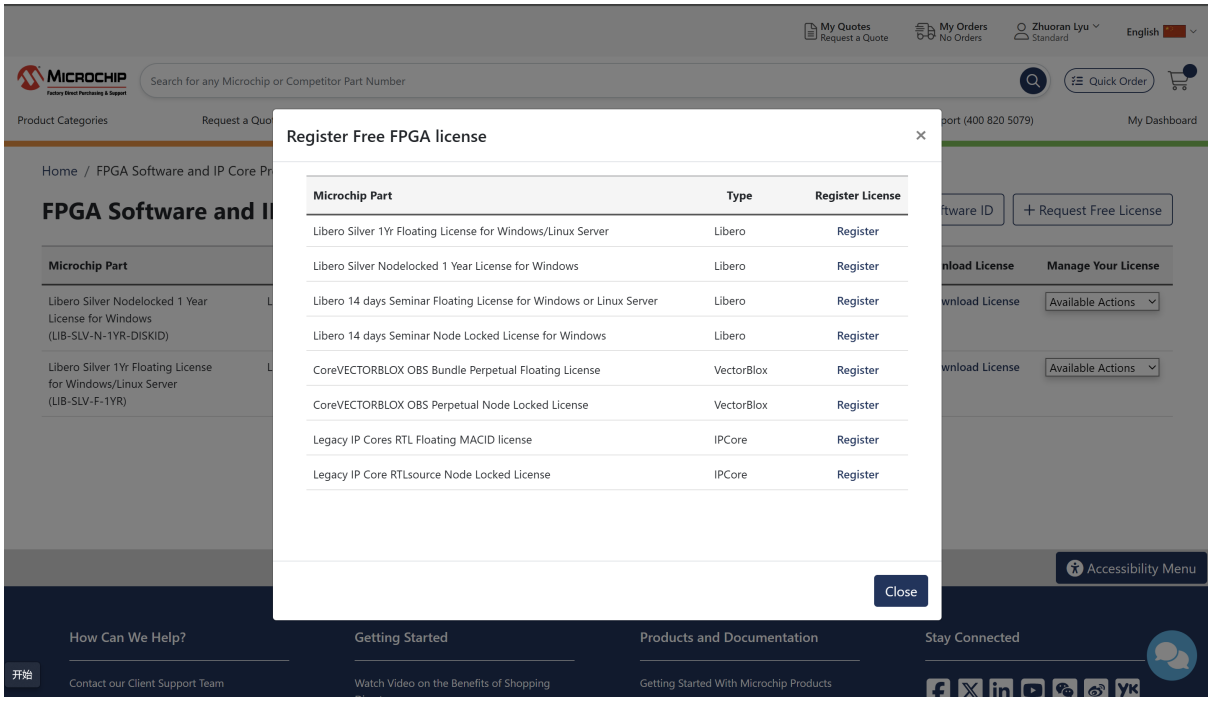

Figure 5: Request License

To improve compatibility, `License.dat` should be stored in a pure ASCII path without spaces or Chinese characters.

```
# Recommended safe storage path in Windows
D:\Microchip\License.dat
```

The system environment variables must then be configured to point to this file. In Windows settings, add or modify the following three variables to point to `D:\Microchip\License.dat`:

- `LM_LICENSE_FILE`
- `SNPSLMD_LICENSE_FILE`
- `SYNPLCTYD_LICENSE_FILE`

After that, Libero SoC can be launched normally.

If license errors still appear, the first things to check are the path to `License.dat`, whether the environment variables are actually effective, whether the DiskID has changed, and whether the VM network adapter MAC address has been reset. To reduce repeated troubleshooting, it is recommended to create a snapshot of the VM when the license is known to work and name it something like `License_Activated_Baseline` so that the stable state can be restored later with a single rollback.

9.1 License Troubleshooting Notes

If Libero still reports authorization errors after startup, the first checks should be:

- Whether the `License.dat` path contains spaces or Chinese characters.
- Whether all three environment variables are actually active.
- Whether the DiskID still matches the one used during application.
- Whether the virtual-machine MAC address has been reset.

It is also advisable to snapshot the VM in a known-good state and name it `License_Activated_Baseline`. If the configuration later becomes unstable, the system can then be rolled back directly to the working state.

10 Libero Post-Installation and Toolchain Setup

After the primary installation of Libero SoC 2025.1 is completed and the `req_to_install.sh` script has been executed, several additional components and environment variables must be configured to establish a fully functional gateware development workflow.

10.1 Install SoftConsole 2022.2

SoftConsole is required for RISC-V software development alongside the FPGA tools. Once the installer is downloaded from the Microchip website, it should be made executable and run:

```
sudo chmod +x Microchip-SoftConsole-v2022.2-RISC-V-747-linux-x64-installer.run
./Microchip-SoftConsole-v2022.2-RISC-V-747-linux-x64-installer.run
```

After finishing the graphical installation, the required dependencies must be installed. A streamlined script is typically provided in the package directory:

```
cd /home/$USER/Microchip/Package-Install
chmod +x install-required-packages.sh
sudo ./install-required-packages.sh
```

To enable non-root users to access the FlashPro programmer, the `udev` rules must be updated and triggered:

```
cd /home/$USER/Microchip/SoftConsole-v2022.2-RISC-V-747/openocd/share/openocd/contrib
sudo cp 60-openocd.rules /etc/udev/rules.d
sudo udevadm trigger
```

10.2 License Configuration and Setup Script

To activate the FPGA tools, a **Libero Silver 1Yr Floating License** must be requested. When entering the machine's MAC address during registration, colons should be omitted (e.g., `123456abcd` instead of `12:34:56:78:ab:cd`).

Once the `License.dat` file is received via email, it should be placed in the `~/Microchip/license` directory and initialized:

```
cd /home/$USER/Microchip/license
chmod +x setup-license-file-2025-1.sh
./setup-license-file-2025-1.sh
```

To ensure the tools are available in every terminal session, the setup script must be appended to the user profile. Open the file with `nano ~/.bashrc` and add the following lines at the end:

```
if [ -f "/home/$USER/Microchip/FPGA-Tools-Setup/setup-microchip-2025-1-tools.
    sh" ]; then
  . "/home/$USER/Microchip/FPGA-Tools-Setup/setup-microchip-2025-1-tools.sh"
fi
```

Additionally, to prevent SSL certificate errors (`ca-certificates` issues) during network operations within the tools, create the required symbolic links:

```
sudo mkdir -p /etc/pki/tls/certs/
sudo ln -s /etc/ssl/certs/ca-certificates.crt /etc/pki/tls/certs/ca-bundle.
    crt
```

10.3 Troubleshooting: License Checkout Errors

When running gateware scripts, if a **License Checkout Error** occurs (e.g., *Cannot locate license file...*), it is often caused by a stalled license manager daemon (`lmgrd`) occupying the default port.

To resolve this, find the process using port `1702` and terminate it:

```
# Identify the process occupying port 1702
lsof -i :1702

# Terminate the lmgrd process using its PID (e.g., if PID is 2582)
sudo kill -9 2582
```

After terminating the stalled process, open a new terminal session and rerun the gateware script. The build flow should now proceed normally.

Troubleshooting: Outdated Dynamic Libraries

It is worth noting that Libero sometimes bundles outdated dynamic libraries. If running certain toolchain scripts results in an error indicating that a required library version cannot be found, a manual override may be necessary. In such cases, locate the corresponding newer dynamic library from your system OS, copy it directly into the specific Libero installation folder, and disable the older bundled version by renaming it (for example, by appending `.bak` to its filename) to prevent conflicts.

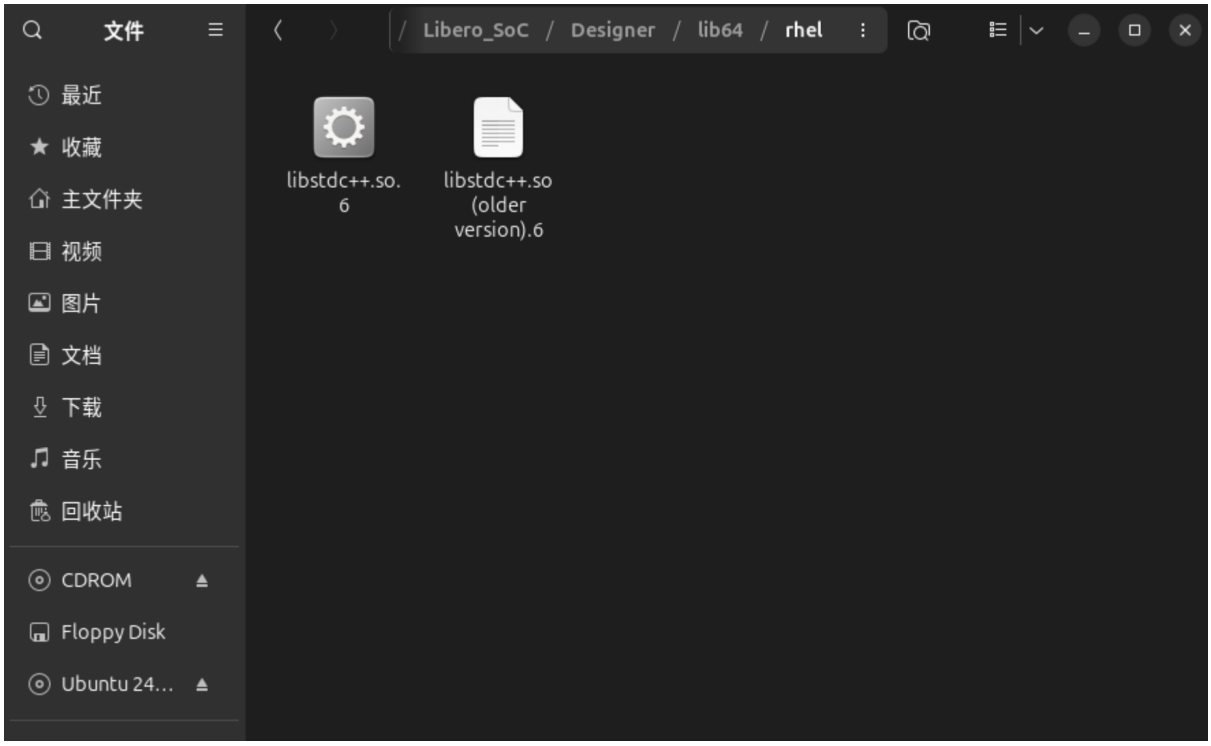

Figure 6: Dynamic Libraries

11 System Validation Checklist

Once these infrastructure layers are in place, the development workflow becomes truly operational. To ensure that the entire process can be reproduced by different people, a layered acceptance check should be performed at the end.

1. Functional Level Checklist:

- The board boots reliably and prints a complete serial log.
- SSH login works normally.
- The board can access the Internet and download dependencies.

- File transfer tools (`scp` and `rsync`) work as expected.

2. Documentation Level Artifacts:

- Image version, flashing-tool version, and parameters.
- Serial log samples, key failure logs, and fixes.
- Network configuration commands and automation scripts.
- VM snapshot naming rules.
- License application and environment-variable instructions.

The report, scripts, and configuration files should also be managed in version control, and every workflow change should be recorded with the date, reason, and verification result. This is the only way to avoid the common problem where the documentation is correct but the actual procedure drifts over time.

3. Known Risks and Mitigations:

- **Network Authentication:** Depends heavily on MAC cloning and can be affected by policy changes.
- **UART Debugging:** Sensitive to human timing; would benefit from automated log collection and keyword alerts.
- **VM Environment:** Depends on the host driver state; best maintained as a clean image plus an automation script and a fixed snapshot.
- **License Mechanism:** Tightly coupled with hardware identity; a unified asset ledger and renewal reminders are advisable.

With these additional safeguards, the current workable process can be elevated into a maintainable, transferable, and scalable engineering workflow.

12 Conclusion

This workflow demonstrates that board bring-up is not a single operation, but a chain of dependent engineering actions that spans hardware validation, image burning, UART diagnosis, bootloader rescue, network adaptation, file transfer, VM isolation, and license activation. Its central value lies in turning scattered setup knowledge into a repeatable process that can be reused across projects.